BECS 22623
Implementation of Numerical Methods using Appropriate software

Solving Ordinary Differential Equations

Department of Mathematics
Faculty of Science
University of Kelaniya

Introduction

A **differential equation** is an equation that includes a variable and one or more of its derivatives.

If a **first derivative is the highest-order** derivative in the equation, the equations is a **first-order differential equation**.

A first-order differential equation can be expressed in the form

$$\frac{dy(t)}{dt} + ay(t) = u(t)$$

The quantity being differentiated y is called the dependent variable. The quantity with respect to which y is differentiated t is called the independent variable.

Introduction

When the function involves **independent variable**, the equation is called an **ordinary differential equation** (or ODE).

This is in contrast to a **partial differential equation** (or PDE) that involves **two or more independent variables**.

We will be solving ordinary differential equations of the form

$$\frac{dy}{dt} = f(t, y)$$

Approach to Euler's method

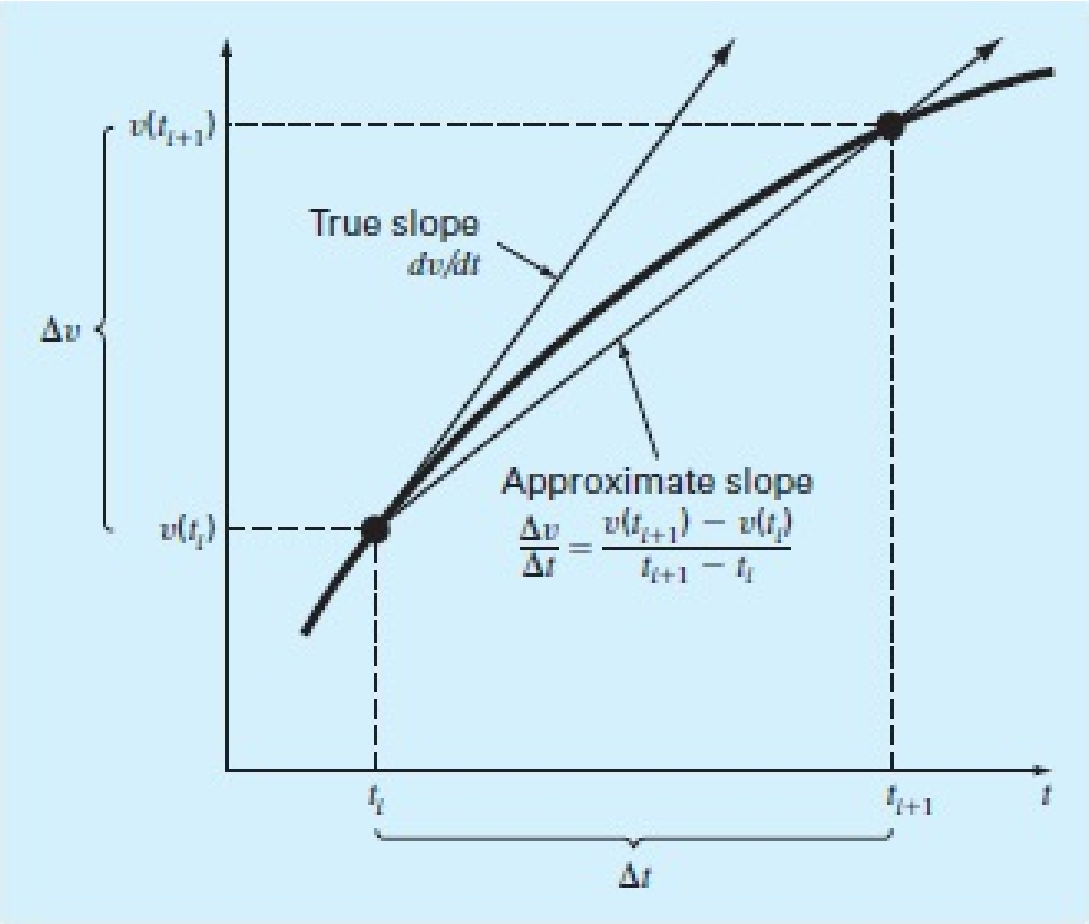


Figure: The use of a finite difference to approximate the first derivative of v with respect to t.

Approach to Euler's method

Observe that

$$\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t}$$

Thus, the equation can be rewritten more concisely as

$$v_{i+1} = v_i + \frac{dv_i}{dt}\Delta t$$

For ordinary differential equations of the form

$$\frac{dy}{dt} = f(t, y)$$

we have the solution in the form

$$y_{i+1} = y_i + \phi h$$

where the slope ϕ is called an increment function. According to this equation, the slope estimate of ϕ is used to extrapolate from an old value y_i to a new value y_{i+1} over a distance h.

Euler's method

$$\frac{dy}{dt} = f(t, y)$$

The first derivative provides a direct estimate of the slope at t_i:

$$\phi = f(t_i, y_i)$$

where $f(t_i, y_i)$ is the differential equation evaluated at t_i and y_i.

This estimate can be substituted into

$$y_{i+1} = y_i + f(t_i, y_i)h$$

This formula is referred to as Euler's method.

A new value of y is predicted using the slope (equal to the first derivative at the original value of t) to extrapolate linearly over the step size h.

Euler's method

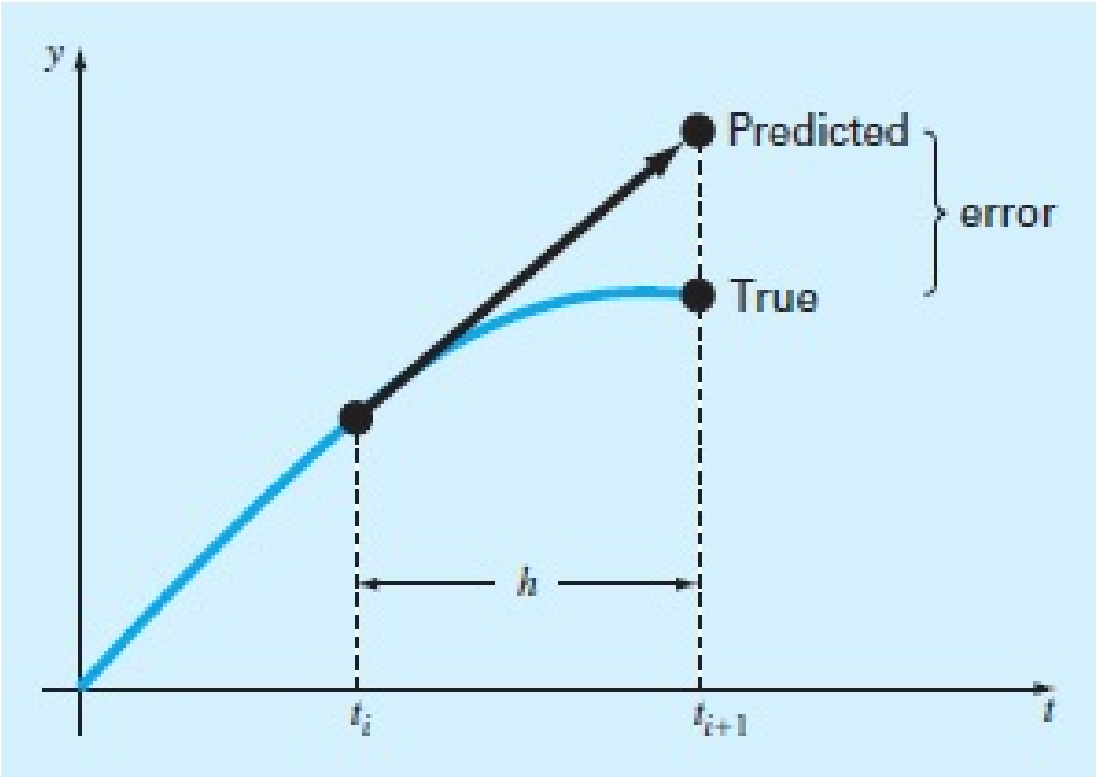


Figure: Euler's method.

Euler's method

Example

Use Euler's method to integrate $y' = 4e^{0.8t} - 0.5y$ from $t = 0$ to 4 with a step size of 1. The initial condition at $t = 0$ is $y = 2$.

Note that the exact solution can be determined analytically as

$$y = \frac{4}{1.3}(e^{0.8t - e^{-0.5t}}) + 2e^{-0.5t}$$

Euler's method

Observe that $f(t, y) = 4e^{0.8t} - 0.5y$.

Implementing Euler's method:

$$y(1) = y(0) + f(0, 2)(1)$$

where $y(0) = 2$ and the slope estimate at $t = 0$ is
$f(0, 2) = 4e^0 - 0.5(2) = 3$.
Therefore, $y(1) = 2 + 3(1) = 5$

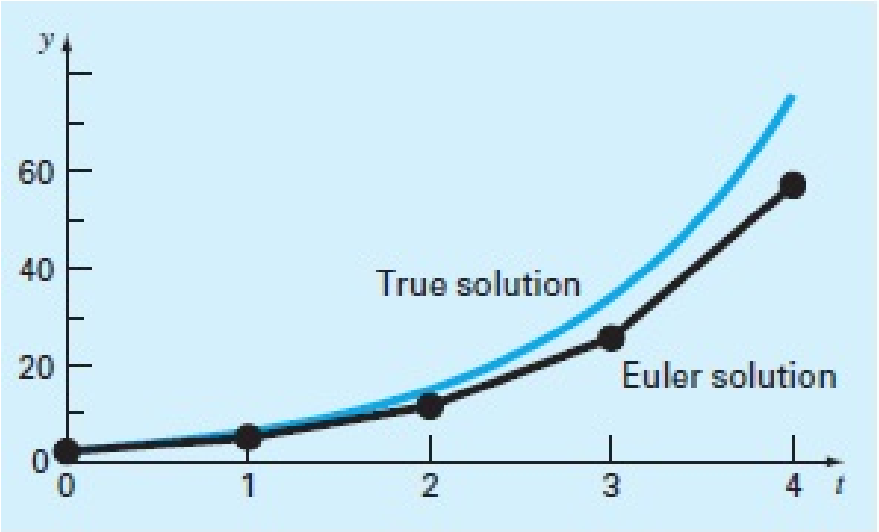
The true solution at $t = 1$ is
$y = \frac{4}{1.3}(e^{0.8(1)-e^{-0.5(1)}}) + 2e^{-0.5(1)} = 6.19463$.

Thus, the percent relative error is $\epsilon_t = \left|\frac{6.19463-5}{6.19463}\right| \times 100\% = 19.28\%$

Euler's method

For the second step: $y(2) = y(1) + f(1,5)(1) = 11.40216$

t	y_{true}	y_{Euler}	$\lvert\varepsilon_t\rvert$ (%)
0	2.00000	2.00000	
1	6.19463	5.00000	19.28
2	14.84392	11.40216	23.19
3	33.67717	25.51321	24.24
4	75.33896	56.84931	24.54



Euler's method | M-file Function: eulode

```
function [t,y] = eulode(dydt,tspan,y0,h,varargin)
% eulode: Euler ODE solver
%   [t,y] = eulode(dydt,tspan,y0,h,p1,p2,...):
%           uses Euler's method to integrate an ODE
% input:
%   dydt = name of the M-file that evaluates the ODE
%   tspan = [ti, tf]  where ti and tf = initial and
%            final values of independent variable
%   y0 = initial value of dependent variable
%   h = step size
%   p1,p2,... = additional parameters used by dydt
% output:
%   t = vector of independent variable
%   y = vector of solution for dependent variable

if nargin<4,error('at least 4 input arguments required'),end
ti = tspan(1);tf = tspan(2);
if ~(tf>ti),error('upper limit must be greater than lower'),end
t = (ti:h:tf)'; n = length(t);
% if necessary, add an additional value of t
% so that range goes from t = ti to tf
if t(n)<tf
  t(n+1) = tf;
  n = n+1;
end
y = y0*ones(n,1); %preallocate y to improve efficiency
for i = 1:n-1 %implement Euler's method
  y(i+1) = y(i) + dydt(t(i),y(i),varargin{:})*(t(i+1)-t(i));
end
```

Euler's method- Problem

Problem

Solve the following initial value problem using Euler's method with $h = 0.5$ and 0.25, over the interval from $t = 0$ to 2 where $y(0) = 1$.

$$\frac{dy}{dt} = yt^3 - 1.5y$$

RUNGE-KUTTA METHODS

Runge-Kutta (RK) methods achieve the accuracy of a Taylor series approach without requiring the calculation of higher derivatives. Many variations exist but all can be cast in the generalized form of Equation:

$$y_{i+1} = y_i + \phi h$$

where ϕ is called an increment function, which can be interpreted as a representative slope over the interval.

The increment function can be written in general form as

$$\phi = a_1 k_1 + a_2 k_2 + ... + a_n k_n$$

RUNGE-KUTTA METHODS

In the increment function a's are constants and the k's are:

$$k_1 = f(t_i, y_i)$$
$$k_2 = f(t_i + p_1 h, y_i + q_{11} k_1 h)$$
.
.
.
$$k_n = f(t_i + p_{n-1} h, y_i + q_{n-1,1} k_1 h + q_{n-1,2} k_2 h + ... + q_{n-1,n-1} k_{n-1} h)$$

Classical Fourth-Order Runge-Kutta Method

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)h$$

where

$$k_1 = f(t_i, y_i)$$

$$k_2 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_1 h\right)$$

$$k_3 = f\left(t_i + \frac{1}{2}h, y_i + \frac{1}{2}k_2 h\right)$$

$$k_4 = f\left(t_i + h, y_i + k_3 h\right)$$

Classical Fourth-Order Runge-Kutta Method

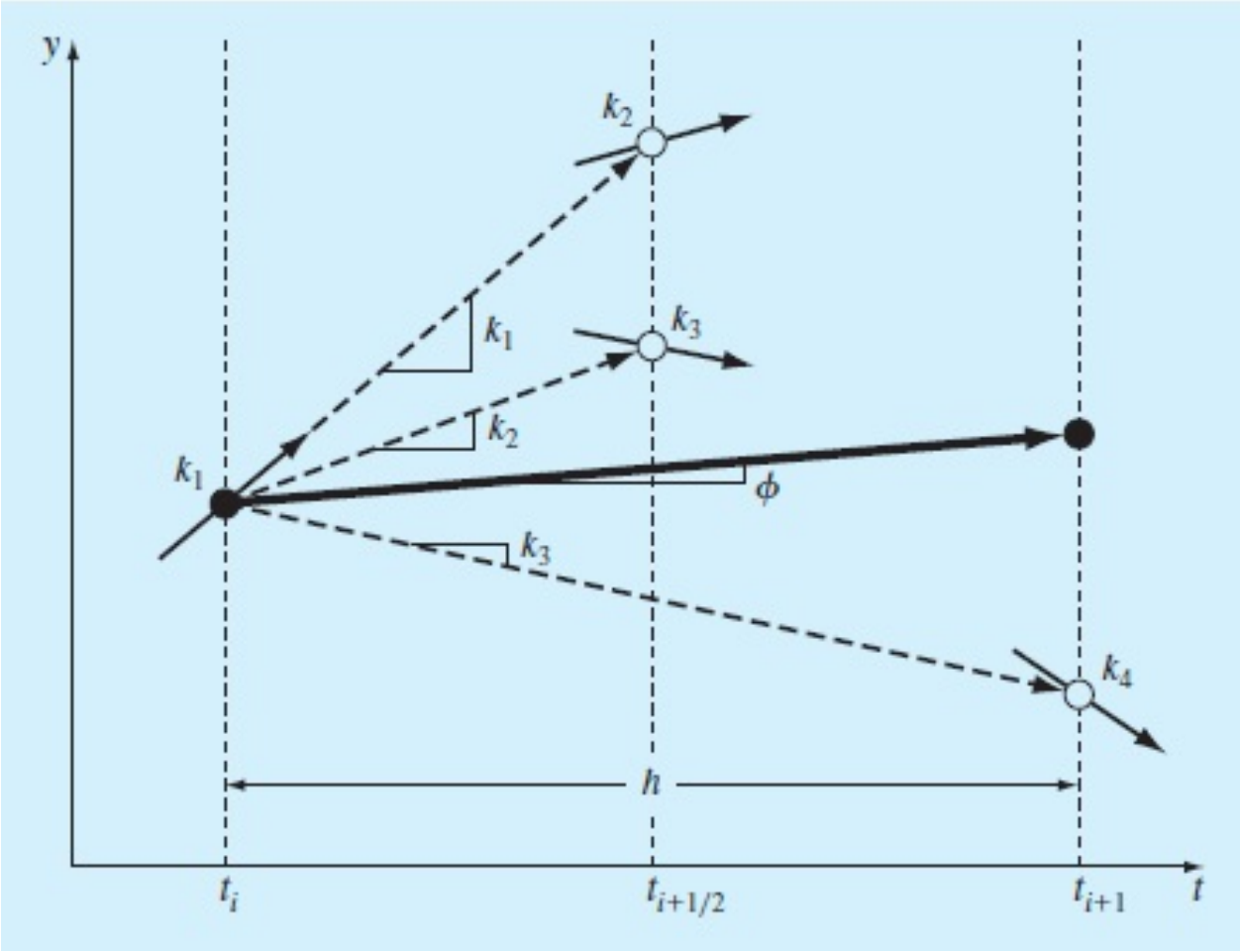


Figure: Graphical depiction of the slope estimates comprising the fourth-order RK method.

Classical Fourth-Order Runge-Kutta Method

Example

Employ the classical fourth-order RK method to integrate $y' = 4e^{0.8t} - 0.5y$ from $t = 0$ to 1 using a step size of 1 with $y(0) = 2$.

For this case, the slope at the beginning of the interval is computed as

$$k_1 = f(0, 2) = 4e^{0.8(0)} - 0.5(2) = 3$$

Classical Fourth-Order Runge-Kutta Method

This value is used to compute a value of y and a slope at the midpoint:

$$y(0.5) = 2 + 3(0.5) = 3.5$$
$$k_2 = f(0.5, 3.5) = 4e^{0.8(0.5)} - 0.5(3.5) = 4.217299$$

This slope in turn is used to compute another value of y and another slope at the midpoint:

$$y(0.5) = 2 + 4.217299(0.5) = 4.108649$$
$$k_3 = f(0.5, 4.108649) = 4e^{0.8(0.5)} - 0.5(4.108649) = 3.912974$$

Next, this slope is used to compute a value of y and a slope at the end of the interval:

$$y(1.0) = 2 + 3.912974(1.0) = 5.912974$$
$$k_4 = f(1.0, 5.912974) = 4e^{0.8(1.0)} - 0.5(5.912974) = 5.945677$$

Finally, the four slope estimates are combined to yield an average slope. This average slope is then used to make the final prediction at the end of the interval.

$$\phi = \frac{1}{6}[3 + 2(4.217299) + 2(3.912974) + 5.945677] = 4.201037$$

$$y(1.0) = 2 + 4.201037(1.0) = 6.201037$$

which compares favorably with the true solution of 6.194631 ($\varepsilon_t = 0.103\%$).

Classical Fourth-Order Runge-Kutta Method | M-File

```
function [tp,yp] = rk4sys(dydt,tspan,y0,h,varargin)
% rk4sys: fourth-order Runge-Kutta for a system of ODEs
%   [t,y] = rk4sys(dydt,tspan,y0,h,p1,p2,...): integrates
%           a system of ODEs with fourth-order RK method
% input:
%   dydt = name of the M-file that evaluates the ODEs
%   tspan = [ti, tf]; initial and final times with output
%                     generated at interval of h, or
%   = [t0 t1 ... tf]; specific times where solution output
%   y0 = initial values of dependent variables
%   h = step size
%   p1,p2,... = additional parameters used by dydt
% output:
%   tp = vector of independent variable
%   yp = vector of solution for dependent variables

if nargin<4,error('at least 4 input arguments required'), end
if any(diff(tspan)<=0),error('tspan not ascending order'), end
n = length(tspan);
ti = tspan(1);tf = tspan(n);
if n == 2
  t = (ti:h:tf)'; n = length(t);
  if t(n)<tf
    t(n+1) = tf;
    n = n+1;
  end
else
  t = tspan;
end
tt = ti; y(1,:) = y0;
np = 1; tp(np) = tt; yp(np,:) = y(1,:);
i=1;
while(1)
  tend = t(np+1);
  hh = t(np+1) - t(np);
```

Classical Fourth-Order Runge-Kutta Method | M-File

```
  if hh>h,hh = h;end
  while(1)
    if tt+hh>tend,hh = tend-tt;end
    k1 = dydt(tt,y(i,:),varargin{:})';
    ymid = y(i,:) + k1.*hh./2;
    k2 = dydt(tt+hh/2,ymid,varargin{:})';
    ymid = y(i,:) + k2*hh/2;
    k3 = dydt(tt+hh/2,ymid,varargin{:})';
    yend = y(i,:) + k3*hh;
    k4 = dydt(tt+hh,yend,varargin{:})';
    phi = (k1+2*(k2+k3)+k4)/6;
    y(i+1,:) = y(i,:) + phi*hh;
    tt = tt+hh;
    i=i+1;
    if tt>=tend,break,end
  end
  np = np+1; tp(np) = tt; yp(np,:) = y(i,:);
  if tt>=tf,break,end
end
```

Classical Fourth-Order Runge-Kutta Method

Problem

Using the fourth-order RK method, solve the following problem over the interval from $x = 0$ to 1 using a step size of 0.25 where $y(0) = 1$.

$$\frac{dy}{dx} = (1 + 4x)\sqrt{y}$$

MATLAB Functions | ode45

MATLAB includes a plethora of functions to solve differential equations under various conditions, but the most all-around useful of them is **ode45**. This function solves first-order ordinary differential equations of the form:

$$y' = f(t, y)$$

The calling sequence for this function is:

```
[t,y] = ode45(odefun_handle,tspan,y0,options)
```

MATLAB Functions | ode45

Calling parameters are:

`odefun_handle`	A *handle* to a function $f(t,y)$ that calculates the derivative y' of the differential equation.
`tspan`	A vector containing the times to integrate. If this is a two-element array `[t0 tend]`, then the values are interpreted as the starting and ending times to integrate. The integrator applies the initial conditions at time `t0` and integrates the equation until time `tend`. If the array has more than two elements, then the integrator returns the values of the differential equation at exactly the specified times.
`y0`	The initial conditions for the variable at time `t0`.
`options`	A structure of optional parameters that change the default integration properties. (We will not use this parameter in this book.)

MATLAB Functions | ode45

The results are:

`t`	A column vector of time points at which the differential equation was solved.
`y`	The solution array. Each row of `y` contains the solutions to all variables at the time specified in the same row of `t`.

MATLAB Functions | ode45

Example

Consider the simple first-order linear time-invariant differential equation

$$\frac{dy}{dt} + 2y = 0$$

with the initial condition $y(0) = 1$.

To use ode45, we must solve this equation for the derivative:

$$\frac{dy}{dt} = -2y$$

MATLAB Functions | ode45

Example

This function could be programmed in MATLAB as follows:

```
>> dydt=@(t,y) -2*y;
>> [t,y]=ode45(dydt,[0 5],1);
```

In approximating this solution, the algorithm ode45 has selected a certain partition of the interval [0, 5], and MATLAB has returned a value of y at each point in this partition.

To obtain solutions at specific times:

```
>> dydt=@(t,y) -2*y;
>> [t,y]=ode45(dydt,[0 1 2 3 4 5],1);
```

or

```
>> [t,y]=ode45(dydt,[0:0.5:5],1);
```

MATLAB Functions | ode45

Problem

The following nonlinear, parasitic ODE was suggested by Hornbeck (1975):

$$\frac{dy}{dt} = 5(y - t^2)$$

If the initial condition is $y(0) = 0.08$, obtain a solution from $t = 0$ to 5, using the MATLAB function **ode45**.

Department of Mathematics

Faculty of Science, University of Kelaniya.